Operating Systems

Abdelrahman Hosny

2026

Faculty of Computers and Information, Assiut University

Contents

01
File Systems

02
Processes

03
Threads

04
Virtual Memory

05
Miscellaneous

06
The Shell

This Lecture

How can we design file systems to manage files on disk, and what are the tradeoffs inherent in designing them? How can we interact with the filesystem in our programs?

Why is this important?

- Helps us understand what filesystems do
- Provides insight into the challenges and tradeoffs in designing large systems
- Shows us how we can directly manipulate files in our programs

Filesystems

A filesystem is the portion of the OS that manages the disk.

A hard drive (or, more commonly these days, flash storage) is persistent storage.

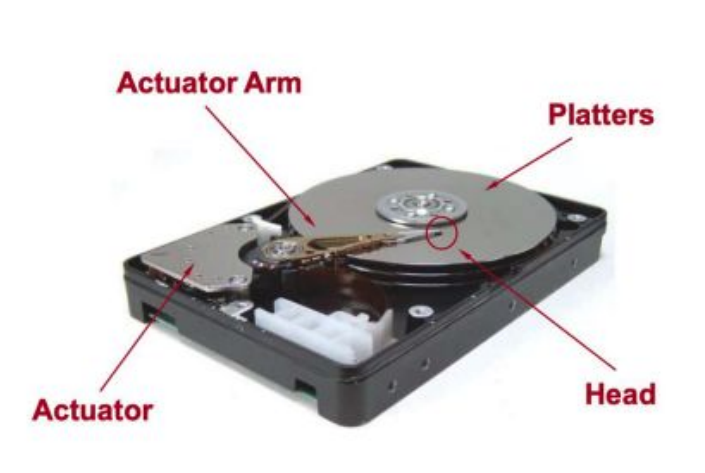
It can store data between power-offs.

Memory (RAM)	**Disk**
- Fast, less space (e.g. 16GB), more expensive to buy per GB. - Byte-addressable: can quickly access any byte of data by address, but not individual bits by address. - Not persistent: cannot store data between power-offs.	- Slower, more space (e.g. 256GB - 1024GB), cheaper to buy per GB - Sector-addressable: cannot read/write just one byte of data; can only read/write “sectors” of data at a time - Persistent: stores data between power-offs

Hard Drives

Magnetic disks (hard drives) have been the standard storage mechanism for files.

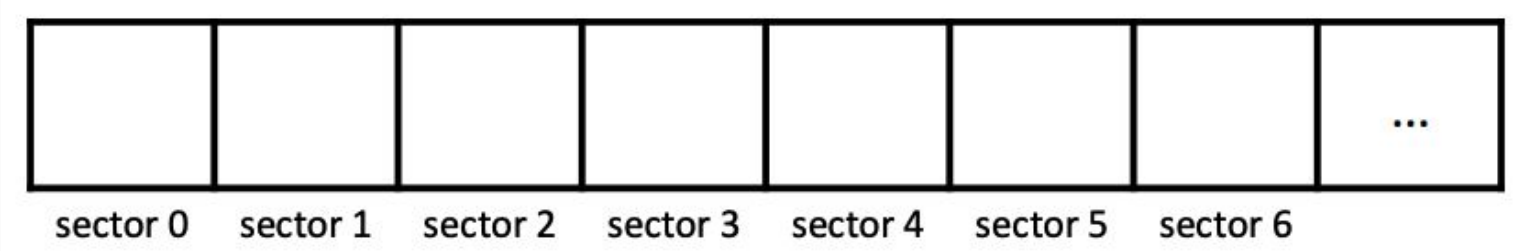
- Spinning, magnetically-coated platters
- Actuator arm positions heads, which can read and write data on the magnetic surfaces
- Moving parts means risk of damage from sudden movement, dust, etc.



Hard Drives

Hard drives have performance characteristics that have a big impact on how we build filesystems.

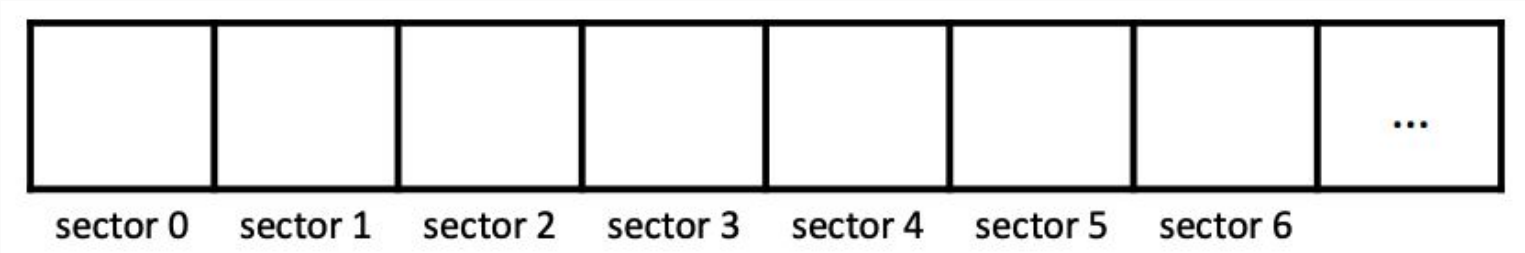
- Reading and writing requires seeking (moving arm to position heads over desired track) and waiting for desired location to pass underneath. Want to minimize this time.
- We can only read data in chunks of sectors.



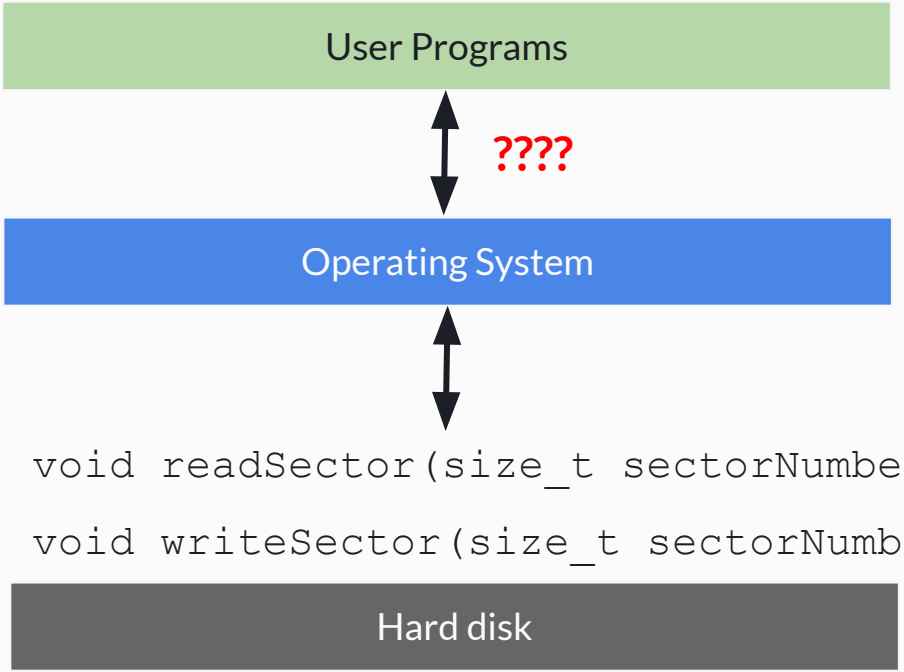
Hard disks are addressable by sectors

If we are the OS, the hard disk creators might provide this API (“application programming interface”) to interface with the disk:

- `void readSector(size_t sectorNumber, void *data);`
- `void writeSector(size_t sectorNumber, const void *data);`



Hard disks are addressable by sectors



This is all we get! We (the OS) must build a filesystem by layering functions on top of these to ultimately allow us to read, write, lookup, and modify entire files

- `void readSector(size_t sectorNumber, void *data);`
- `void writeSector(size_t sectorNumber, const void *data);`

Filesystem Functionality

We want to read/write data on disk and have them persist even when the device is off.

- creating a new file on disk
- looking up the location of a file on disk
- reading/editing all or part of an existing file from disk – e.g., sequential/random access
- creating folders on disk
- getting the contents of folders on disk
- ...

Filesystems

Functions for user programs to read/write files

Filesystem

`readSector` and `writeSector`

Filesystem Challenges

Problems addressed by modern file systems:

- Disk space management:
  - Fast access to files (minimize seeks)
  - Sharing space between users
  - Efficient use of disk space
- Naming: how do users select files?
- Reliability: information must survive OS crashes and hardware failures.
- Protection: isolation between users, controlled sharing.

Flash Storage

Recently, flash storage (“SSD”) has become more popular and commonplace, especially with the growth in mobile devices.

- Much faster (100x faster access), but more expensive
- No moving parts, so more reliable
- Issues with wear-out; once a chunk of the drive has been erased many times (~100k), it no longer stores info reliably.
- **Typically, still use an interface of reading/writing in units of sectors**

Any questions?
No stupid question!

Let's discuss any doubts or concerns.
If something comes up later, contact abdelrahman@aun.edu.eg

Methods for Storing Files

- Contiguous Allocation
- Linked Files
- Windows FAT
- Multi-level indexes

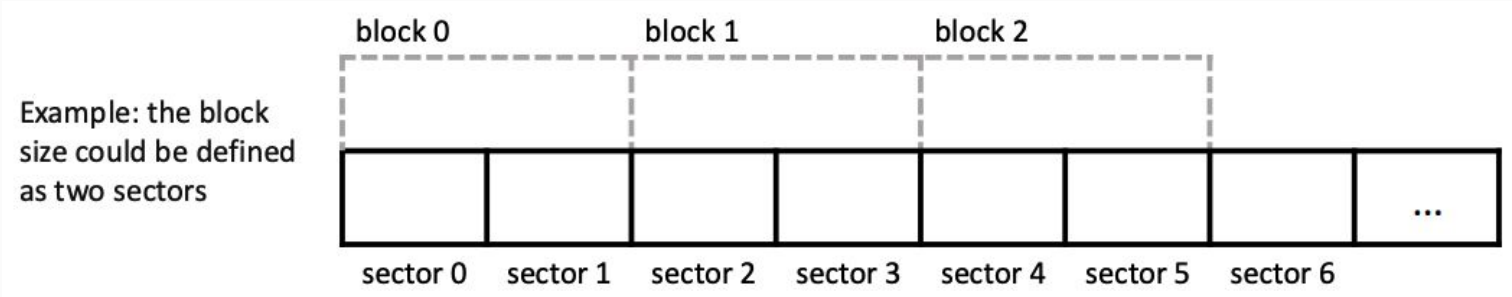
Sectors and Blocks

A filesystem generally defines its own unit of data, a "block," that it reads/writes at a time

- "Sector" = hard disk storage unit
- "Block" = filesystem storage unit (1 or more sectors) - software abstraction

Pros of larger block size? Smaller block size?

- more efficient I/O if larger, but less internal fragmentation if small



What data are we storing?

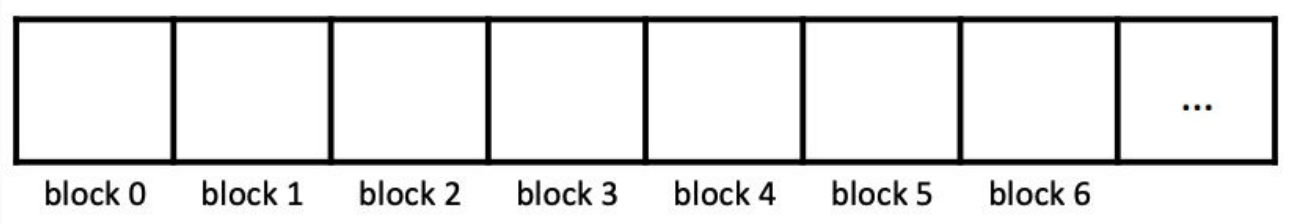
Two types of data we will be working with:

1. **file payload data** - contents of files (e.g. text in documents, pixels in images)
2. **file metadata** - information about files (e.g. name, type, extension, size)

Key insight: both must be stored on the hard disk. Otherwise, we will not have it across power-offs! (e.g. without storing metadata we would lose all filenames after shutdown). This means some blocks must store data other than payload data.

Contiguous Allocation

Should we store files contiguously on disk? What would it look like if we did?



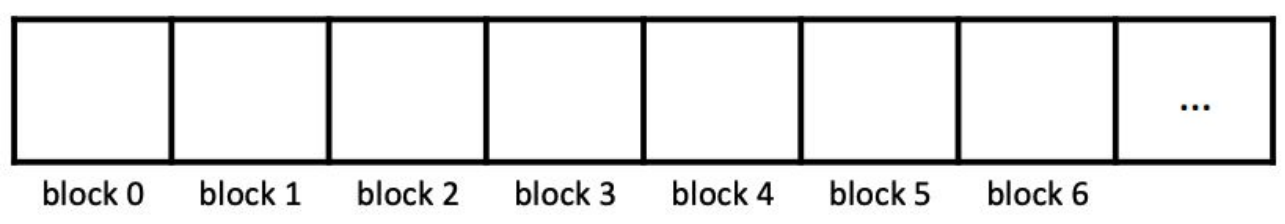
Called contiguous allocation: allocate a file in one contiguous group of blocks

- For each file, keep track of the number of its first block and its length
- Keep a free list of unused areas of the disk
- For our purposes, we won't allocate partial blocks (only give out full blocks)

Example: IBM OS/360

Contiguous Allocation - Advantages?

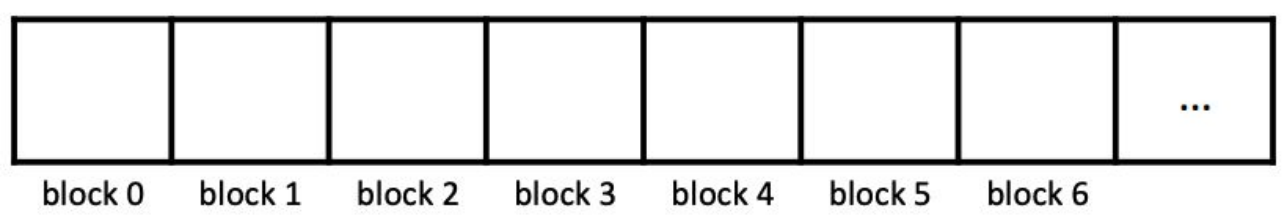
Should we store files contiguously on disk? What would it look like if we did?



1. simple
2. can read sequentially or easily jump to any location in file (“random access”)
3. all data in one place (few seeks)

Contiguous Allocation - Disadvantages?

Should we store files contiguously on disk? What would it look like if we did?



1. hard to grow files
2. hard to lay out files on disk; we may not be able to squeeze a new file in a block of free space (external fragmentation – no single space is large enough, but enough aggregate space is available)
3. Also internal fragmentation (space allocated is larger than needed) if we allocate in units of blocks

Fragmentation

Internal Fragmentation

space allocated for a file is larger than what is needed. A file may not take up all the space in the blocks it's using. E.g. block = 512 bytes, but file is only 300 bytes. (you could share blocks between multiple files, but this gets complex)

External Fragmentation (issue with contiguous allocation)

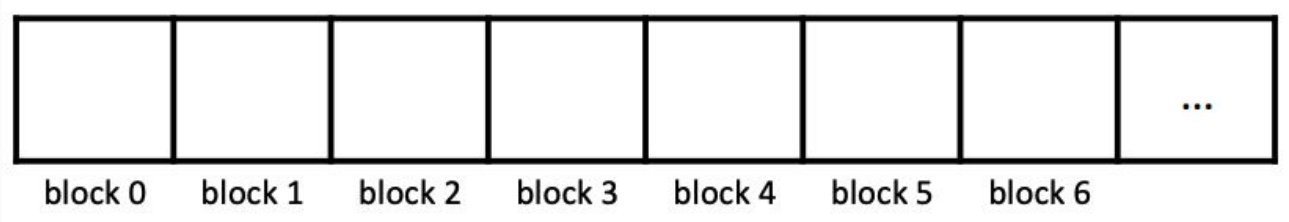
no single space is large enough to satisfy an allocation request, even though enough aggregate free disk space is available

Non-Contiguous Allocation

How would we organize data in our hard disk if it's not contiguous?

Problem

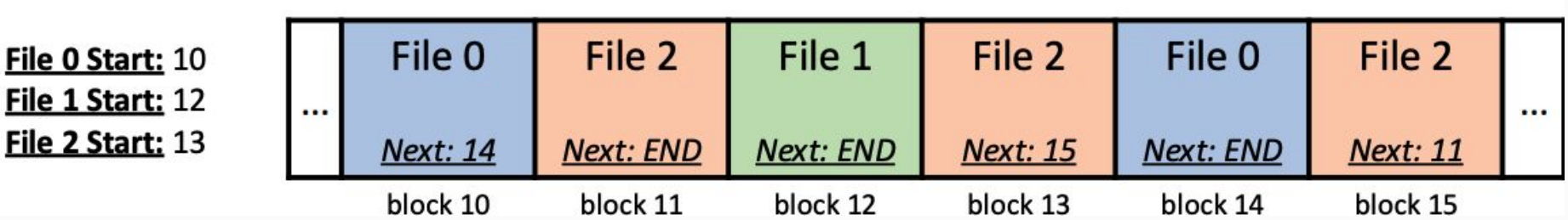
We need to know what blocks are associated with what files



Files as a linked list

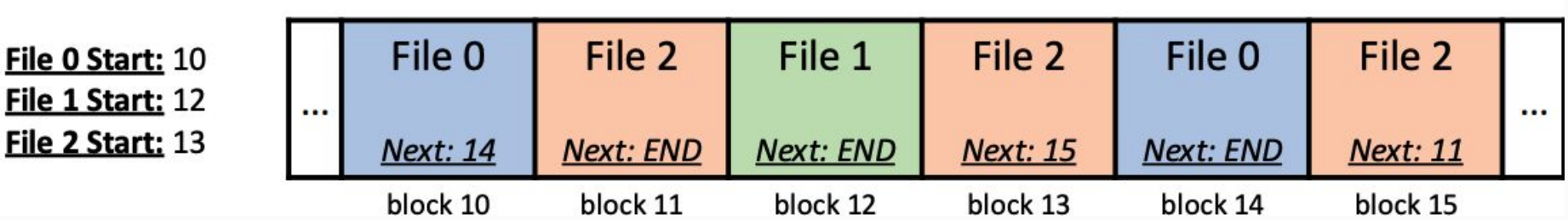
Each block contains file data as well as the location of the next block

- For each file, keep track of the number of its first block in separate location
- Approximate examples: TOPS-10, Xerox Alto



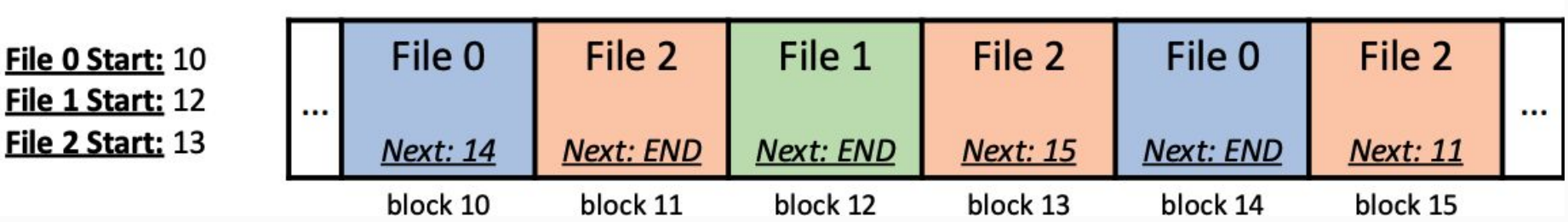
Files as a linked list - Advantages?

- Easy to grow files
- Easier to fit files in available space – less fragmentation
- Still supports simple sequential access



Files as a linked list - Disadvantages?

- Can't easily jump to any arbitrary location in the file
- Data scattered throughout disk (more seeks vs. contiguous allocation)



Files as a lookup table

Store the links in one big table in memory

- Windows (DOS) FAT: like linked allocation, except links aren't in blocks, they are in a "file allocation table" in memory and disk

In-Memory File Allocation Table

	...
10	14
11	END
12	END
13	15
14	END
15	11
	...

File 0 Start: 10
File 1 Start: 12
File 2 Start: 13

...	File 0	File 2	File 1	File 2	File 0	File 2	...
	Next: 14	*Next: END*	*Next: END*	*Next: 15*	*Next: END*	*Next: 11*	
	block 10	block 11	block 12	block 13	block 14	block 15	

Files as a lookup table

Store the links in one big table in memory

- Windows (DOS) FAT: like linked allocation, except links aren't in blocks, they are in a "file allocation table" in memory and disk
- Still keep track of each file's first block
- Still used today for flash sticks, digital cameras, many embedded devices

In-Memory File Allocation Table

	...
10	14
11	END
12	END
13	15
14	END
15	11
	...

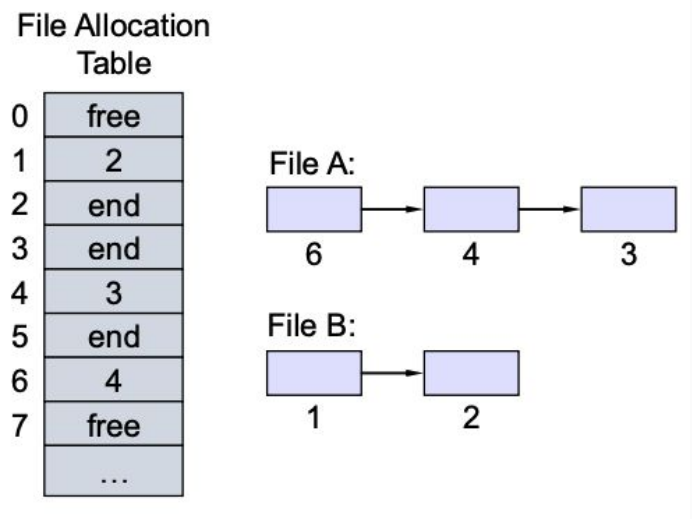
File 0 Start: 10
File 1 Start: 12
File 2 Start: 13

...	File 0	File 2	File 1	File 2	File 0	File 2	...
	block 10	block 11	block 12	block 13	block 14	block 15	

Files as a lookup table - Advantages?

Store the links in one big table in memory

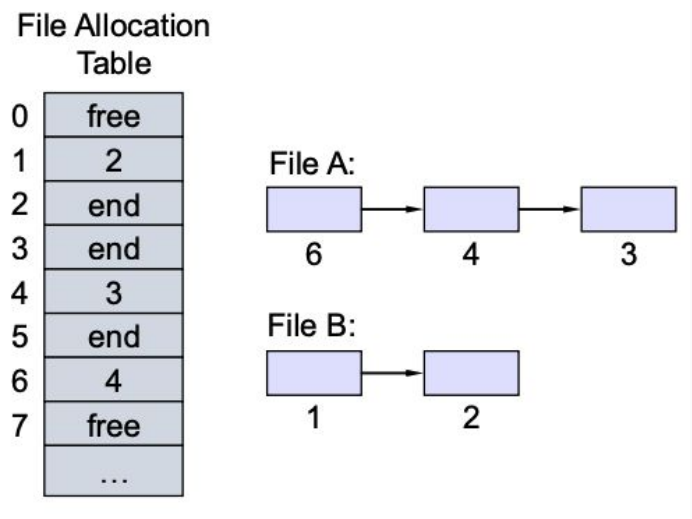
- Can more quickly jump to various locations in a file
- Still supports easy sequential access



Files as a lookup table - Disadvantages?

Store the links in one big table in memory

- Data scattered throughout disk (more seeks vs. contiguous allocation).
- Still need to jump through table to get to an arbitrary location in the file.
- Must store table in memory.



Files with better lookup

Store all the block numbers for a file in order. That way we could quickly jump to any point in the file.

File 0: 10, 14

File 1: 12

File 2: 13, 15, 11

In-Memory File Allocation Table

	...
10	14
11	END
12	END
13	15
14	END
15	11
	...

File 0 Start: 10
File 1 Start: 12
File 2 Start: 13

...	File 0	File 2	File 1	File 2	File 0	File 2	...
	block 10	block 11	block 12	block 13	block 14	block 15	

Files with better lookup

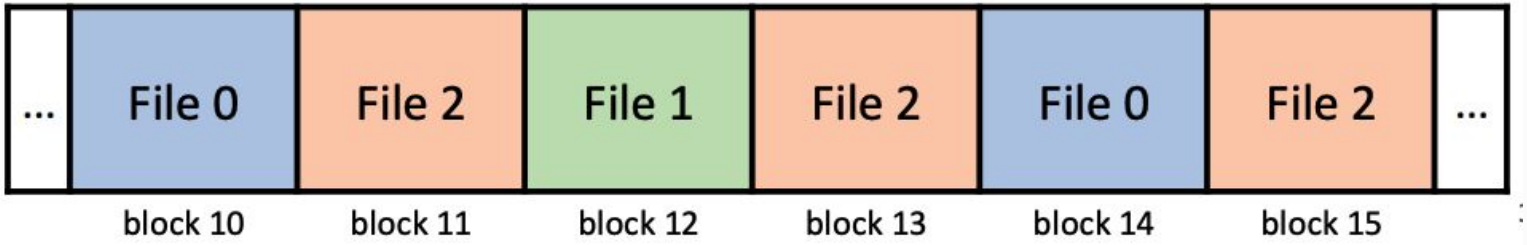
Multi-level indexes: store all block numbers for a given file (but how?)

- Example: 4.3 BSD Unix, Unix V6 Filesystem (~1975)
- More modern ext2 and ext3 Linux file systems based on this idea; Windows NTFS also uses a tree-based structure, though slightly different

File 0: 10, 14

File 1: 12

File 2: 13, 15, 11



Summary so far

- Linked lists → in-memory lookup table (still traversing) → store all block references

Store all block references? What data structure should we use?

Next time

- Linked lists → in-memory lookup table (still traversing) → store all block references

Store all block references? What data structure should we use?

Trees!

- Unix file system
- inodes
- Let's break the system

Any questions?
No stupid question!

Let's discuss any doubts or concerns.
If something comes up later, contact abdelrahman@aun.edu.eg